

Bahan Ajar Lengkap: Struktur Data, Fungsi, dan Pemrograman Berorientasi Objek di MQL5

Penyusun: Tim Tanggal: 2025

Daftar Isi

3.1	Struktur dan Union	3
3.1.1	Definisi Struktur	3
3.1.2	Fungsi (Metode) dalam Struktur	4
3.1.3	Penyalinan Struktur	4
3.1.4	Konstruktor dan Destruktor	5
3.1.5	Penyusunan Struktur (Packing)	6
3.1.6	Komposisi dan Pewarisan pada Struktur	6
3.2	Pemrograman Berorientasi Objek (OOP)	7
3.2.1	Abstraksi	7
3.2.2	Enkapsulasi dan Hak Akses	7
3.2.3	Pewarisan (Inheritance)	8
3.2.4	Polimorfisme	8
3.2.5	Komposisi dalam Desain Kelas	9
3.2.6	Definisi Kelas	9
3.2.7	Hak Akses (Access Specifiers)	10
3.2.8	Konstruktor Kelas	11
3.2.9	Destruktor Kelas	11
3.2.10	Penunjuk <i>this</i> (Self-Reference)	12
3.2.11	Pewarisan Lanjutan	12
3.2.12	Alokasi Dinamis (operator new dan delete)	13
3.2.13	Pointer dan Referensi	13
3.2.14	Metode Virtual (Pengikatan Dinamis)	14
3.2.15	Anggota Statik (Static Members)	15
3.2.16	Catatan: Jenis Bersarang, Namespace, dan Pemisahan Deklarasi	15
3.2.17	Catatan: Kelas Abstract dan Interface	16
3.2.18	Catatan: Overloading Operator dan Casting	16
3.3	Template	16
3.3.1	Header Template	16
3.3.2	Prinsip Umum Template	17

3.1 Struktur dan Union

3.1.1 Definisi Struktur

Struktur (struct) dalam MQL5 adalah tipe data terkomposit yang terdiri dari satu atau lebih variabel (field) dengan tipe dasar atau tipe buatan pengguna. Tujuan utama struktur adalah **mengelompokkan data yang terkait secara logis dalam satu wadah**. Sebagai contoh, jika kita mempunyai banyak parameter yang secara alami saling terkait (misalnya, parameter-parameter konfigurasi suatu perhitungan), lebih mudah membungkusnya dalam satu struktur daripada melewatkannya satu per satu.

Definisi struktur menggunakan kata kunci struct, diikuti nama struktur, dan diakhiri dengan blok { ... } berisi deklarasi field serta tanda titik koma. Misalnya, struktur Settings dapat didefinisikan sebagai berikut :

```
struct Settings
{
    datetime start;      // tanggal mulai analisis
    int    barNumber;    // jumlah bar (histori kutipan)
    ENUM_APPLIED_PRICE price; // tipe harga yang digunakan
    int    components;   // jumlah komponen parameter
};
```

Penjelasan baris per baris:

- struct Settings { ... }; – Mendefinisikan tipe baru bernama Settings sebagai kumpulan beberapa field.
- Di dalam kurung kurawal, setiap baris mendeklarasikan satu field: datetime start; menyatakan field start bertipe datetime; seterusnya untuk barNumber, price, dan components.
- Setelah }, perhatikan adanya ; karena definisi tipe dianggap pernyataan penuh.

Setelah struktur dideklarasikan, kita dapat menggunakan tipe Settings seperti tipe built-in lainnya. Misalnya, menambahkan deklarasi variabel:

```
Settings s;
```

Maka s akan memiliki field s.start, s.barNumber, s.price, s.components yang bebas diisi. Setiap instansi struktur memiliki salinan tersendiri dari field-field tersebut . Untuk mengakses atau mengubah field, gunakan operator titik .: misalnya s.start = D'2021.01.01'; . Sebagai catatan, Ciri khas: anggota (field) struktur secara default bersifat public, sehingga bisa diakses di mana saja (berbeda dengan kelas yang defaultnya private). Contoh penggunaan dan inisialisasi struktur:

```
void OnStart()
{
    Settings s;
    s.start    = D'2021.01.01';
    s.barNumber = 1000;
    s.price    = PRICE_CLOSE;
    s.components = 8;
```

```
// Atau menggunakan inisialisasi agregat:
Settings s2 = { D'2021.01.01', 1000, PRICE_CLOSE, 8 };
}
```

Pada contoh di atas, `s2 = { ... }` menginisialisasi langsung semua field sesuai urutan deklarasi . Jika jumlah nilai lebih sedikit, sisa field diinisialisasi ke nilai nol default.

3.1.2 Fungsi (Metode) dalam Struktur

MySQL5 memungkinkan fungsi didefinisikan di dalam struktur sebagai metode (method). Fungsi-fungsi ini dapat mengolah data field struktur secara langsung tanpa perlu dereferensi. Mendefinisikan fungsi di dalam struktur membuatnya lebih mudah memproses isi struktur secara khusus . Sebagai contoh, misalnya kita punya struktur `Result` yang menyimpan beberapa hasil perhitungan, dan ingin menambah metode `print()` untuk mencetak isinya ke log:

```
struct Result
{
    double probability;
    double coef ;
    int    direction;
    string status;

    void print()
    {
        Print(probability, " ", direction, " ", status);
        ArrayPrint(coef);
    }
};
```

Penjelasan kode di atas:

- `struct Result { ... }`; sama seperti definisi struktur biasa.
- Di dalamnya, selain field data, kita menambahkan `void print() { ... }`. Ini adalah metode anggota struktur.
- Metode `print()` dapat mengakses field `probability`, `direction`, `status`, `coef` secara langsung (tanpa `s.` karena konteks `this`) . Misalnya, `Print(probability, ...)` mencetak nilai `probability`.
- Untuk memanggil metode, kita cukup menggunakan operator titik pada variabel struktur:
`Result r = calculate(settings);`
`r.print();` // Memanggil metode `print` pada instansi `r`

Metode dalam struktur adalah cara yang rapi untuk menggabungkan data dan fungsi terkait, mendekatkan logika pemrosesan bersama data tersebut.

3.1.3 Penyalinan Struktur

Struktur yang memiliki tipe sama dapat dengan mudah disalin ke struktur lain menggunakan operator (`=`). Penyalinan ini dilakukan secara *field-by-field*. Contoh:

```
Result r = calculate(s);
r.print();
// Anggap hasil:
// 0.5 1 ok
```

```
// 1.00000 2.00000 3.00000
```

```
Result r2;  
r2 = r;  
r2.print();  
// Output r2 akan sama dengan r (data disalin)
```

Artinya, setiap field *r* disalin ke *r2* satu per satu . Penting: Struktur berbeda dengan nama berbeda **tetap dianggap jenis yang berbeda**, meski memiliki field identik; hanya struktur tipe yang sama yang dapat di-copy sepenuhnya. Jika ingin menyalin dari satu tipe ke tipe lain yang terkait (misalnya melalui pewarisan struktur), hanya field yang cocok akan disalin.

3.1.4 Konstruktor dan Destruktor

Seperti di kelas, struktur di MQL5 dapat memiliki fungsi khusus yaitu **konstruktor** dan **destruktor**. Konstruktor adalah metode khusus yang bernama sama dengan struktur dan tanpa tipe pengembalian, yang dipanggil otomatis saat instansi struktur dibuat. Misalnya, kita dapat menambahkan dua konstruktor ke struktur *Result*: satu tanpa parameter dan satu dengan parameter string untuk mengatur status:

```
struct Result  
{  
    double probability;  
    double coef ;  
    int    direction;  
    string status;  
  
    // Konstruktor default  
    void Result()  
    {  
        status = "ok";  
    }  
    // Konstruktor dengan parameter  
    void Result(string s)  
    {  
        status = s;  
    }  
};
```

Penjelasan: Metode *Result()* dan *Result(string s)* berperan sebagai konstruktor. MQL5 menganggap fungsi yang namanya sama dengan struktur dan tidak memiliki tipe sebagai konstruktor . Konstruktor ini akan otomatis men-set nilai awal pada field yang diperlukan saat objek dibuat. Jika **tidak ada konstruktor** yang didefinisikan, kompiler akan membuat konstruktor default bawaan (yang menetapkan nilai nol *default* pada *field-field*).

Sebaliknya, **destruktor** ditulis dengan tanda tilde ~ di depan nama struktur/class. Destruktor dipanggil saat objek akan dihancurkan, misalnya saat variabel keluar scope atau delete digunakan. Destruktor tidak mengembalikan nilai dan tidak dapat di-overload (hanya satu destruktur). Contoh:

```

struct Result
{
    // ... (fields dan metode lain)

    ~Result()
    {
        // Kode pembersihan jika perlu (misal menutup file)
        Print("Result object destroyed");
    }
};

```

Jika destruktur tidak didefinisikan atau kosong, MQL5 akan secara otomatis membersihkan jenis-jenis field yang memerlukan seperti string, array dinamis, dll. Sebagai catatan, dalam MQL5 semua destruktur dianggap virtual secara implisit (mirip C++) , namun sintaksnya sama seperti C++.

3.1.5 Penyusunan Struktur (Packing)

MQL5 menyediakan pengaturan penyusunan memori (alignment) struktur melalui kata kunci pack. Dengan #pragma pack(size) atau penamaan struct Identifier pack(size), kita dapat menentukan perataan byte untuk struktur. Misalnya pack(1) memaksa jarak byte minimum (byte alignment) sama dengan sizeof(byte). Hal ini membantu kompatibilitas dengan struktur di DLL atau format file tertentu . Contoh penggunaan _offsetof untuk mengetahui offset field:

```
Print(offsetof(Result, status)); // Menampilkan offset byte dari field status
```

Serta, perlu diingat bahwa urutan penyusunan field memengaruhi ukuran struktur; umumnya bidang terbesar diletakkan di depan untuk meminimalkan padding .

3.1.6 Komposisi dan Pewarisan pada Struktur

Struktur di MQL5 dapat **mengandung** struktur lain sebagai field (komposisi), maupun **mewarisi** dari struktur lain. Sebagai contoh komposisi:

```

struct Point { double x, y; };
struct Line {
    Point start;
    Point end;
};

```

Di sini Line “memiliki” dua Point. Akses ke field dilakukan bertingkat: line.start.x, line.end.y, dsb. Jika struktur yang didefinisikan hanya digunakan di sini, bisa juga dideklarasikan di dalam struktur lain:

```

struct Line
{
    struct Point { double x, y; } start;
    int id;
};

```

Pewarisan struktural mirip seperti di kelas: dengan : setelah nama struktur, kita dapat memperluas struktur lain. Misalnya:

```
struct Base { double X, Y; };  
struct Derived : Base {  
    int code;  
};
```

Mendeklarasikan `Derived : Base` berarti `Derived` mewarisi field `X`, `Y` dari `Base`. Memori `Derived` ditata dengan `Base` di awal, diikuti field baru `code`. Dari sisi lay out, struktur terwarisi seperti ini memiliki susunan memori yang setara dengan mendefinisikan field `X`, `Y` secara langsung, sehingga ukuran dan offset bisa sama . Namun secara tipe, `Derived` tetap berbeda dengan `Base`. Menyalin antar tipe berbasis pewarisan diperbolehkan: *member parent* yang sama akan disalin, sisanya diabaikan atau dibiarkan tak tersentuh sesuai konteks . Sebagai catatan, jika `parent` hanya punya konstruktor parameter, konstruktor `child` harus menginisiasi `parent` melalui *initializer list*.

3.2 Pemrograman Berorientasi Objek (OOP)

3.2.1 Abstraksi

Abstraksi dalam OOP berarti menyediakan antarmuka yang mudah digunakan tanpa mengungkap detail implementasi. Dalam praktiknya, kita merancang kelas sebagai “kotak hitam” dengan serangkaian fungsi (method) yang membentuk antarmuka (interface) dan field yang menyimpan state. Pengguna kelas tidak perlu tahu detail bagaimana fungsi bekerja, hanya butuh tahu fungsi apa yang tersedia. Hal ini memungkinkan perubahan algoritma internal tanpa mengubah cara penggunaan kelas. Contohnya, sebuah fungsi `draw()` di kelas `Shape` dapat diimplementasikan berbeda untuk `Rectangle` atau `Circle`, tetapi dari luar pengguna memanggil fungsi yang sama . Dalam istilah struktur-program, semua fungsi yang dijabarkan dalam konteks kelas disebut metode. Metode-metode ini, bersama field yang menyimpan data, membentuk antarmuka kelas. Abstraksi menyederhanakan pemrograman dengan memusatkan kompleksitas di dalam kelas dan menyajikan fungsi simpel di luar kelas.

3.2.2 Enkapsulasi dan Hak Akses

Enkapsulasi berarti menyembunyikan detail implementasi kelas agar aman dari intervensi luar. Ide sederhananya mirip perangkat elektronik: pengguna boleh memakai fungsi utama, tapi tidak sembarang membuka komponennya agar tidak rusak. MQL5 menggunakan tiga level hak akses seperti C++: `private` (default kelas), `protected`, dan `public` .

- 1) **Private:** Anggota kelas hanya terlihat dari dalam definisi kelas itu sendiri. Ini melindungi data internal kelas. Misalnya pada class `Shape`, field `x` dan `y` bisa dibuat `private` sehingga `shape.x` tidak bisa diakses di luar.
- 2) **Protected:** Anggota dapat diakses oleh kelas itu sendiri dan kelas turunannya. Memudahkan pewarisan tanpa mengekspos anggota ke semua bagian program.
- 3) **Public:** Anggota terlihat di mana saja, sebagai antarmuka utama kelas. Fungsi `draw()` atau `toString()` biasanya `public` agar pemakai kelas dapat memanggilnya.

Sebagai ilustrasi, jika kita mencoba mengakses field `private` di luar kelas, kompiler akan mengeluarkan error “cannot access private member” . Enkapsulasi memaksa

programmer memikirkan skema akses: apapun yang tidak diperlukan untuk pengguna kelas sebaiknya disembunyikan (*private*) . Sebagai prinsip desain, pembuat kelas disarankan menggunakan hak akses seketat mungkin, sehingga meminimalkan kesalahan tak terduga saat penggunaan ulang kode.

3.2.3 Pewarisan (Inheritance)

Pewarisan memungkinkan sebuah kelas baru mengambil sifat (*field* dan *metode*) dari kelas lain. Kelas yang diwarisi disebut *parent* atau *base class*, sedangkan kelas penerus disebut *child* atau *derived class*. Dengan pewarisan, kita dapat menggunakan ulang kode: kelas anak mewarisi semua implementasi kelas induk tanpa menulis ulang, hanya menambah atau memperbaiki yang perlu . Misalnya, jika terdapat kelas *CShape* umum dengan atribut *type*, *xpos*, *ypos*, kita dapat membuat *CCircle* : *public CShape* yang secara otomatis punya *type*, *xpos*, *ypos* dan hanya menambahkan *radius* sendiri.

Pewarisan bersifat hierarkis: kita dapat menurunkan satu atau beberapa tingkatan. Sebagai contoh, *Square* bisa mewarisi dari *Rectangle*, yang mewarisi dari *Shape*. Hubungan ini dikenal sebagai “is-a”: **Square is-a Rectangle** dan juga **Square is-a Shape**. Perubahan apa pun di kelas basis (misalnya menambah *field* baru) akan tampak di semua kelas turunannya, membuat pemeliharaan kode lebih efisien . Namun, pemrogram harus berhati-hati merancang pewarisan agar hanya dilakukan jika memang memenuhi konsep “is-a”.

Syntax pewarisan di MQL5 sama seperti C++:

```
class Base { /* ... */ };  
class Derived : public Base { /* ... */ };
```

Keyword *public* di atas mendefinisikan akses pewarisan (bisa juga *protected* atau *private*). Pewarisan *public* artinya anggota *public* dan *protected* dari *Base* akan mempertahankan visibilitas yang sama di *Derived* . Contoh pewarisan tipe *struct* sudah dibahas sebelumnya pada bagian 3.1.6.

3.2.4 Polimorfisme

Polimorfisme (“banyak bentuk”) adalah kemampuan satu antarmuka umum dijalankan dengan cara berbeda tergantung tipe objek yang sebenarnya. Konsep ini erat kaitannya dengan pewarisan: jika beberapa kelas turunan berbagi metode dengan nama yang sama (dan *signature* identik), panggilan metode pada kelas turunan dapat menghasilkan perilaku yang berbeda. Contohnya, dalam sistem grafik, kita bisa memiliki kelas dasar *Shape* dengan metode *draw()*, yang di-*override* di tiap turunan:

```
class Shape { public: virtual void draw(); };  
class Circle : public Shape { public: void draw() override { /* gambar lingkaran */ } };  
class Square : public Shape { public: void draw() override { /* gambar persegi */ } };
```

Kemudian kita bisa menyimpan berbagai *Shape** dalam satu array dan memanggil *draw()* secara seragam. Polimorfisme muncul karena pemanggilan *shape->draw()* akan menyesuaikan implementasi berdasarkan jenis objek yang sebenarnya (*Circle* vs *Square*) . Tanpa polimorfisme, kita perlu memanggil fungsi berbeda sesuai tipe, yang membuat kode tak terstruktur. Dengan polimorfisme, panggilan tunggal *draw* dalam *loop* akan **menghasilkan** gambar bentuk yang tepat sesuai objeknya .

Dalam istilah yang lebih luas, polimorfisme berarti sebuah operasi (*metode*) dapat

diterapkan pada tipe data yang berbeda. Ini memungkinkan kita menulis kode yang lebih umum dan fleksibel. Pada bagian berikut, akan dibahas bagaimana MQL5 mengikat metode secara **statis** (compile time) atau **dinamis** (runtime) menggunakan kata kunci virtual dan override.

3.2.5 Komposisi dalam Desain Kelas

Dalam merancang sistem OOP, kita sering memecah entitas kompleks menjadi kelas-kelas yang lebih sederhana. Tiga hubungan utama antar-objek dalam desain OOP adalah **komposisi**, **agregasi**, dan **asosiasi**.

- 1) **Komposisi** (komposisional agregasi) – hubungan whole-part yang kuat. Sebuah objek “memiliki” objek lain secara penuh, sehingga bagian itu tidak dapat eksis terpisah. Contoh: sebuah Car memiliki sebuah Engine sebagai komponen penting; engine tidak berarti tanpa mobil. Penghancuran Car otomatis menghancurkan Engine-nya. Implementasi MQL5: dalam kelas, mendefinisikan field berjenis objek. Misalnya:

```
class Car {  
    Engine engine; // komposisi: mobil PUNYA engine  
    // ...  
};
```

- 2) **Agregasi** – hubungan whole-part yang lebih longgar. Sebuah objek memiliki referensi ke objek lain yang bisa bersifat opsional. Objek bagian dapat eksis mandiri, bahkan bisa dimiliki oleh beberapa “pemilik”. Contoh: sebuah StudentGroup memiliki daftar Student. Siswa bisa keluar masuk grup tanpa menghancurkan objek Student.
- 3) **Asosiasi** – hubungan lemah di mana dua objek berinteraksi atau menggunakan satu sama lain. Misalnya Printer dan Computer; komputer “menggunakan” printer, tetapi keduanya eksis mandiri.

Memahami ketiga jenis hubungan ini membantu merancang kelas yang sesuai. Selain itu, prinsip-prinsip desain OOP umum (seperti DRY, SRP, OCP) juga perlu diperhatikan untuk struktur kelas yang baik. Sebagai contoh, prinsip DRY (Don't Repeat Yourself) menyarankan agar bagian kode umum diangkat ke kelas induk jika memungkinkan, menghindari duplikasi.

3.2.6 Definisi Kelas

Kelas adalah cetak biru (blueprint) untuk membuat objek (instans kelas). Sintaks definisinya adalah:

```
class ClassName [: modifier_access BaseClassName ...]  
{  
    // deklarasi anggota (field dan method)  
};
```

Sama seperti struktur, kita gunakan kata kunci class diikuti nama dan blok {} berisi anggota. Jika ada pewarisan, ditulis setelah : (misal : public BaseClass). Akhiri dengan ; seperti biasanya. Contoh:

```
class Shape  
{  
    int x, y;          // Koordinat pusat (field)
```

```
color backgroundColor; // Warna isi (field)
};
```

Dalam contoh, Shape adalah kelas baru dengan dua field x,y dan satu field backgroundColor . Sama seperti struktur, kita bisa membuat variabel bertipe Shape, misalnya Shape s;. Namun perlu diingat, anggota kelas defaultnya private jika tidak ada specifier akses . Jadi s.x atau s.y di luar kelas Shape akan error karena sifat private-nya, ini menunjukkan penerapan enkapsulasi . Jika kita ingin field tersebut terbuka, harus ditandai public: . Misalnya:

```
class Shape {
public:
    int x, y;          // sekarang dapat diakses di luar
    color backgroundColor;
};
```

Selain field, biasanya kita menambahkan metode (fungsi anggota) dalam kelas. Contoh metode sederhana:

```
class Shape {
private:
    int x, y;
    color backgroundColor;
public:
    string toString() {
        return(StringFormat("Shape at (%d,%d) color %s", x, y,
            colorToString(backgroundColor)));
    }
    void draw() { /* implementasi gambar nanti */ }
};
```

Pada contoh di atas, toString() dan draw() didefinisikan di dalam kelas untuk melayani kebutuhan output/menampilkan objek. Seperti pada struktur, metode dalam kelas memungkinkan pengolahan internal objek dengan cara terstruktur.

3.2.7 Hak Akses (Access Specifiers)

Seperti dibahas, MQL5 menggunakan tiga specifier utama dalam kelas: private, protected, dan public. Specifier ini menentukan aksesibilitas anggota berikutnya hingga ditemui specifier baru. Contoh:

```
class CExample {
    int a;          // private by default
public:
    int b;          // publik: dapat diakses dari mana saja
protected:
    int c;          // protected: dapat diakses dalam kelas dan turunannya
private:
    int d;          // private: hanya diakses dalam kelas ini
};
```

Secara default, jika tidak ditulis apa-apa (seperti pada `int a`; di atas), anggota adalah `private`. Prinsip enkapsulasi menuntut agar detail yang tidak perlu diakses luar diletakkan di kelompok `private`. Pengguna kelas hanya melihat apa yang dideklarasikan `public`. Ini mencegah pemrogram secara tak sengaja merusak state internal kelas. Sebagai catatan dari teori, pengaturan akses yang tepat sangat penting; lebih baik memakai aturan `private` sebanyak mungkin dan hanya membuka yang dibutuhkan.

3.2.8 Konstruktor Kelas

Konstruktor kelas mirip dengan struktur: adalah metode khusus dengan nama sama dengan kelas dan tanpa `return type`. Tujuannya menginisialisasi objek baru. Kelas dapat memiliki beberapa konstruktor (overloading) dengan parameter berbeda. Saat objek dibuat, konstruktor yang sesuai (berdasarkan argumen) akan dipanggil otomatis. Misalnya:

```
class Point {
public:
    int x, y;
    Point()      // konstruktor default
    {
        x = 0; y = 0;
    }
    Point(int a, int b) // konstruktor parametrik
    {
        x = a; y = b;
    }
};
```

Dalam contoh, `Point()` inisialisasi ke (0,0), sedangkan `Point(3,4)` menghasilkan objek (3,4). Bila tidak ada konstruktor eksplisit, kompiler otomatis membuat konstruktor default (nilai nol). Konstruktor kelas biasanya diletakkan di bagian `public`. Kita juga dapat menggunakan initializer list dalam konstruktor untuk langsung menginisialisasi field sebelum tubuh fungsi dijalankan (efisiensi dan keharusan bagi `const` atau field tanpa konstruktor default):

```
Point(int a, int b) : x(a), y(b) { }
```

3.2.9 Destruktor Kelas

Destruktor kelas memiliki nama sama dengan kelas dan diawali tanda tilde `~`, tanpa parameter dan tanpa `return value`. Destruktor dipanggil otomatis saat objek dihancurkan (misalnya saat scope berakhir atau `delete` dipanggil). Fungsinya untuk membersihkan sumber daya (memori, file, dsb). Contoh:

```
class Logger {
public:
    Logger() { /* buka file log */ }
    ~Logger() {
        // tutup file log, dsb.
    }
};
```

Jika destruktur tidak didefinisikan, kompiler akan otomatis membersihkan field-field seperti string, dynamic array, dll . Di MQL5, semua destruktur dianggap virtual (saling terkait dalam v-table), tapi kita tidak perlu menuliskan kata kuncinya . Umumnya destruktur dibuat public, tapi bisa juga private/protected untuk mencegah pembuatan objek secara otomatis di luar (misal pada pola factory).

3.2.10 Penunjuk *this* (Self-Reference)

Di dalam setiap metode kelas, terdapat penunjuk khusus *this* yang otomatis menunjuk ke objek saat ini. *this* adalah pointer implisit ke instans kelas yang sedang diproses . Kita hanya dapat menggunakan *this* dalam metode non-statik. Contohnya:

```
class Shape {
public:
    int x, y;
    void moveX(int dx) {
        x += dx;        // sama dengan this->x += dx;
    }
    void moveY(int dy) {
        this.y += dy;    // menggunakan this secara eksplisit
    }
};
```

Kedua cara di atas identik. Penggunaan *this* eksplisit berguna bila terjadi name hiding: jika ada variabel lokal atau parameter dengan nama sama seperti field, maka penulisan *this->field* memastikan yang diakses benar-benar field objek . Selain itu, *this* bisa dikembalikan dari metode untuk memperbolehkan method chaining. Misalnya:

```
class Shape {
public:
    Shape* setColor(const color c) {
        backgroundColor = c;
        return this;
    }
    Shape* moveX(int dx) {
        x += dx;
        return this;
    }
};
...
s.setColor(clrRed).moveX(10);
```

Metode-metode di atas mengembalikan *this*, sehingga dapat dirangkai dalam satu baris. Hal ini meningkatkan keterbacaan saat mengubah beberapa properti secara beruntun.

3.2.11 Pewarisan Lanjutan

Meskipun telah dijelaskan konsep pewarisan dasar, beberapa hal perlu diulang dalam konteks kelas:

- Saat mendefinisikan kelas turunan, kita menyertakan kelas induk di deklarasinya. Contohnya, `class Rectangle: public Shape { ... }`; menyatakan Rectangle mewarisi dari

Shape.

- Dalam pewarisan, kita dapat menambah akses anggota (misal mengubah protected menjadi public) jika perlu, tetapi biasanya kita mempertahankan yang sama.
- Kita dapat memanggil metode kelas induk dari kelas turunan menggunakan operator ::. Contoh: di kelas Square kita bisa memanggil implementasi draw() milik Rectangle secara eksplisit: Rectangle::draw();. Hal ini berguna jika ingin memanfaatkan logika dasar dari induk sebelum menambahkan perilaku baru.
- Melakukan pewarisan panjang (multi-level) juga diperbolehkan: misalnya class Square : public Rectangle { ... };, lalu class Rectangle : public Shape { ... };. Object Square kemudian memiliki semua anggota Shape dan Rectangle, dengan kemungkinan override yang berlapis.

Secara umum, pewarisan mempermudah reuse, namun harus dirancang matang agar hierarchy tetap logis (“is-a”).

3.2.12 Alokasi Dinamis (operator new dan delete)

Biasanya, objek dibuat secara otomatis (misal sebagai variabel lokal dalam fungsi) dan dihancurkan saat keluar scope. Namun, dalam banyak kasus (misal membuat daftar objek dinamis sesuai input pengguna), kita butuh alokasi objek secara dinamis. MQL5 mendukung ini melalui operator new (untuk alokasi) dan delete (untuk dealokasi). Contoh membuat objek dinamis:

```
Rectangle *pr = new Rectangle(100, 200, 50, 75, clrBlue);
```

- new Rectangle(...) akan memanggil konstruktor Rectangle dengan argumen yang diberikan, dan mengembalikan pointer Rectangle* ke objek yang baru dibuat.
- Variabel pr di atas adalah pointer ke objek Rectangle. Penting: hanya mendeklarasikan Rectangle *pr; tidak membuat objek, perlu new.
- Untuk mengakses anggota objek dari pointer, gunakan operator -> (seperti pr->draw()). Ada juga (*pr).field jika perlu dereferensi manual.
- Nilai default pointer sebelum diinisialisasi adalah NULL. Setelah new, pr menjadi pointer valid ke objek dinamis.

Jika objek dinamis sudah tidak diperlukan, panggil delete pr; untuk melepaskan memori. Jika dilupakan, akan terjadi memory leak dan MQL5 akan memperingatkan sejumlah objek tak terhapus saat program berakhir. Setelah delete pr;, pointer pr menjadi tidak valid. Sebaiknya selalu cek apakah pointer valid sebelum delete, misalnya dengan fungsi CheckPointer() bawaan MQL5. Contoh:

```
if(CheckPointer(pr) == POINTER_DYNAMIC) {  
    delete pr;  
}
```

Penggunaan alokasi dinamis memberikan fleksibilitas manajemen memori, tetapi menuntut tanggung jawab programmer untuk menghapus objek secara eksplisit agar tidak terjadi pemborosan memori.

3.2.13 Pointer dan Referensi

Di MQL5, pointer untuk objek kelas berbeda dengan pointer di C++: sebenarnya

berupa descriptor (nomor unik) yang digunakan terminal untuk mengenali objek. Meskipun demikian, model penggunaannya mirip: kita menyimpan hasil new dalam variabel pointer dan mengakses anggota melalui ->. Pointer otomatis (non-dinamis) juga ada misal pada field kelas jika kita menyimpan objek lain.

Sementara itu, objek kelas dan struktur **selalu dilewatkan sebagai referensi** jika menjadi parameter fungsi. Artinya, jika kita memanggil func(obj), yang terjadi adalah func(ReferenceToObj). MQL5 tidak mengizinkan parameter bertipe objek untuk dilewatkan by-value. Sebagai gantinya, gunakan tanda ampersand: void func(Shape &obj). Contoh: pada struktur Settings, fungsi update yang diperbarui adalah double calculate(Settings &settings);. Dalam hal ini settings adalah referensi ke objek asli, bukan salinan. Ini juga berlaku untuk pengembalian referensi.

Jika kita meletakkan simbol & pada variabel objek dalam konteks lain (misal Print(&s);), MQL5 menganggapnya sebagai pointer ke objek, bukan reference parameter seperti dalam deklarasi fungsi. Oleh karena itu Print(&s); mencetak nomor objek (pointer) bukan isi objek. **Intinya:**

- 1) Objek kelas/struktur tidak pernah secara otomatis disalin (kecuali jika kita eksplisit menyalinnya seperti dibahas di 3.1.3).
- 2) Pengiriman ke fungsi harus lewat referensi.
- 3) Pointer adalah nomor unik, perlu new untuk membuat objeknya.

3.2.14 Metode Virtual (Pengikatan Dinamis)

Secara default, penentuan metode mana yang dipanggil oleh sebuah objek terjadi pada saat kompilasi (static binding). Contohnya:

```
class Shape {
public:
    void draw() { Print("Base draw"); }
};
class Rectangle : public Shape {
public:
    void draw() { Print("Drawing rectangle"); }
};
...
Shape *shape = new Rectangle();
shape->draw(); // Output: "Base draw"
```

Walaupun shape menunjuk ke Rectangle, yang dipanggil adalah Shape::draw() karena method tidak virtual – pemilihan dibuat berdasarkan tipe pointer saat compile-time. Hal ini menyulitkan implementasi polimorfisme: misalnya dalam loop, setiap shape->draw() memanggil versi base, bukan yang spesifik turunan.

Untuk mengaktifkan dynamic dispatch (pilih metode pada runtime sesuai objek), MQL5 mendukung metode virtual mirip C++. Menandai metode di kelas dasar dengan virtual memberi tahu kompiler untuk menggunakan virtual table (v-table) saat pemanggilan. Contoh:

```
class Shape {
public:
    virtual void draw() { Print("Base draw"); }
};
class Rectangle : public Shape {
public:
```

```
void draw() override { Print("Drawing rectangle"); }
};
```

- virtual sebelum tipe return metode di Shape menandakan metode ini bersifat virtual.
- Di turunan Rectangle, kita menuliskan draw() dengan override (opsional tetapi disarankan) untuk menegaskan bahwa ini override metode virtual dasar.
- Kini Shape *shape = new Rectangle(); shape->draw(); akan memanggil Rectangle::draw() dan mencetak "Drawing rectangle". Kompilasi menghasilkan **dynamic binding** menggunakan table virtual.

MQL5 juga memperbolehkan metode virtual mengembalikan pointer ke kelas (covariant return type) selama jenis kembaliannya lebih spesifik di turunan, contohnya virtual Shape *setColor(...) di kelas dasar dan virtual Rectangle *setColor(...) override di turunan. Dengan metode virtual, kita dapat memanggil turunan yang sesuai di runtime, memungkinkan polimorfisme yang sejati.

3.2.15 Anggota Statik (Static Members)

Kelas/struktur di MQL5 dapat memiliki anggota statik yang dilabeli dengan static. Anggota statik adalah data atau fungsi yang dibagi oleh seluruh instansi kelas, bukan milik tiap objek. Mereka seolah-olah berada di scope kelas, bukan di objek. Misalnya, kita ingin menghitung total objek Shape yang dibuat. Maka kita dapat menambahkan:

```
class Shape {
private:
    static int count; // penghitung statik
public:
    Shape() { ++count; }
    static int getCount() { return count; }
};
int Shape::count = 0; // definisi inisialisasi statik di luar kelas
```

Penjelasan:

- static int count; (pada bagian private) berarti hanya ada satu variabel count untuk semua Shape.
- Konstruktor menambah count saat objek baru dibuat.
- static int getCount() adalah metode statik yang mengembalikan nilai count.
- Di luar kelas kita harus mendefinisikan int Shape::count = 0; untuk mengalokasikan memori dan inisiasinya.

Akses ke anggota statik dilakukan dengan operator resolusi konteks :: contohnya Shape::getCount(). Bahkan kita dapat memanggilnya melalui objek (misal shape->getCount()), tapi lebih baik menggunakan ClassName::member untuk kejelasan, karena akses statik tidak memerlukan objek nyata. Anggota statik sering dipakai untuk menghitung total objek, menyimpan konstanta kelas, atau fungsi utilitas kelas.

3.2.16 Catatan: Jenis Bersarang, Namespace, dan Pemisahan Deklarasi

MQL5 juga mendukung kelas/struct bersarang (di dalam kelas lain atau namespace) untuk mengorganisir tipe. Misalnya, kita bisa mendefinisikan tipe helper di dalam kelas lain. Operator :: (scope resolution) digunakan untuk merujuk pada tipe dalam namespace atau kelas

tertentu. Selain itu, deklarasi kelas dan definisinya dapat dipisah (seperti header/implementasi) menggunakan file .mqh untuk deklarasi dan .mq5 untuk definisi.

3.2.17 Catatan: Kelas Abstract dan Interface

MQL5 tidak memiliki konsep interface terpisah, tetapi kita bisa meniru kelas abstrak (yang berisi metode virtual murni) dengan membuat metode virtual tanpa implementasi (tidak boleh ada di MQL5 – jadi lebih sering kita cukup punya kelas dasar yang tak berguna sendiri). Namun, tipe pointer dan virtual method dapat berperan sebagai “antarmuka” serupa interface.

3.2.18 Catatan: Overloading Operator dan Casting

MQL5 memungkinkan overloading operator tertentu (seperti +, ==, dsb) dengan mendefinisikan fungsi operator+, operator== dalam kelas. Ini mempermudah operasi antar objek seperti pada tipe built-in. Selain itu, konversi objek dapat dilakukan dengan dynamic_cast (ke pointer) saat menggunakan pewarisan polimorfik, serta casting ke/ dari void* (pointer generik). Fitur final dan delete pada pewarisan juga didukung untuk mengakhiri rantai override.

3.3 Template

3.3.1 Header Template

Template di MQL5 adalah mekanisme pembuatan fungsi atau kelas generik. Sebuah template mendefinisikan algoritma umum dengan parameter tipe, dari mana kompiler kemudian membuat instansiasi khusus berdasar penggunaan tipe nyata. Syntax dasar header template adalah:

```
template <typename T>
```

atau untuk beberapa parameter:

```
template <typename T1, typename T2>
```

Misalnya, untuk membuat fungsi generik Max yang bekerja pada berbagai tipe numerik:

```
template <typename T>
T Max(T a, T b)
{
    return (a > b) ? a : b;
}
```

Baris template <typename T> memberitahu kompiler bahwa T adalah tipe yang akan ditentukan nanti. Definisi lengkapnya (fungsi atau kelas) ditulis setelah header template. Saat kita memanggil Max(3,5), kompiler otomatis menggantikan T dengan int sehingga menghasilkan int Max(int, int). Untuk Max(3.5, 2.1) hasilnya double Max(double, double). Dengan template kita tidak perlu menulis berulang untuk tiap tipe. Catatan: **MQL5 tidak mendukung semua fitur template C++** (contoh: template non-tipe, parameter default, parameter variadic, spesialisasi parsial), sehingga penggunaannya lebih sederhana.

3.3.2 Prinsip Umum Template

Template sangat membantu saat kita memiliki fungsi dengan logika sama tapi parameter tipe berbeda. Contoh klasik: fungsi Max. Tanpa template, kita harus menulis ulang fungsi untuk setiap tipe:

```
double Max(double a, double b) { return a > b ? a : b; }
int Max(int a, int b) { return a > b ? a : b; }
datetime Max(datetime a, datetime b) { return a > b ? a : b; }
```

Semua implementasi serupa. Dengan template kita cukup menulis satu:

```
template<typename T>
T Max(T a, T b)
{
    return (a > b) ? a : b;
}
```

Penjelasan:

- `template<typename T>` mendefinisikan template dengan parameter tipe T.
- `T Max(T a, T b)` adalah deklarasi fungsi; T di sini akan digantikan oleh tipe aktual saat penggunaan.
- `return (a > b) ? a : b;` logika yang sama untuk semua tipe.
- Saat kita memanggil `Max(10,20)` di MQL5, kompiler membuat versi `int Max(int,int);`; memanggil `Max(1.5, 2.7)` menghasilkan `double Max(double,double)`.

Format umum template class pun mirip, menggunakan `template<typename ...>` sebelum deklarasi kelas. Dengan template, kita dapat membuat struktur data generik (seperti `template<typename T> class Stack { T *data; ... };`) atau fungsi matematis, menghindari duplikasi kode. Harap dicatat bahwa MQL5 membatasi template: misalnya tidak ada default type parameter dan tidak ada spesialisasi parsial. Meski begitu, untuk keperluan umum (template fungsi dan template kelas sederhana) fitur ini sangat berguna.

Ringkasan

- Struktur (struct): wadah data terkomposit. Dapat memiliki fungsi/method sendiri, konstruktor, destruktur. Menyusun ulang field (packing) dan pewarisan antar struktur juga didukung .
- Kelas (class): cetak biru objek yang mirip struktur tetapi dengan enkapsulasi (private default), bisa diwarisi, dan mendukung metode, konstruktor/destruktur, operator overloading, dll .
- Fungsi/Metode: di dalam struktur/kelas, dijalankan pada objek. Dapat di-override di kelas turunannya. Fungsi virtual (dengan kata kunci virtual) memastikan pemanggilan metode berdasarkan tipe objek sebenarnya (dynamic dispatch).
- Pewarisan: mewarisi anggota dari kelas induk untuk reuse kode. Hubungan “is-a”. Anggota induk dapat diakses di kelas turunan sesuai aturan akses (public/protected) .
- Enkapsulasi: proteksi data internal kelas dengan hak akses (private/protected/public). Default kelas adalah private (field harus di-expose secara eksplisit dengan public jika diperlukan) .
- Abstraksi: interface kelas (metode publik) dipisahkan dari implementasi internal. Pengguna kelas cukup melihat antarmuka tanpa mengetahui logika detail .
- Pointer dan Referensi: objek kelas/struktur di-passing by-reference, bukan by-value. Pointer objek adalah ID internal, diakses dengan new/delete untuk alokasi dinamis.
- Anggota Statis: data/fungsi milik kelas, bukan objek. Disebut dengan ClassName::member. Berguna untuk variabel bersama atau metode utilitas umum.
- Template: definisi generik fungsi atau kelas agar mudah digunakan untuk berbagai tipe. Dideklarasikan dengan template <typename T>. Kompilator membuat instansi spesifik saat diperlukan.

Catatan penting: Di MQL5 (sama seperti C++), objek kelas besar tidak pernah dikopi secara otomatis saat dilewatkan ke fungsi; selalu gunakan referensi. Pastikan members yang perlu diakses dari luar dideklarasikan public. Manfaatkan konstruktor/destruktur untuk inisialisasi dan pembersihan sumber daya. Gunakan virtual untuk metode yang dipoliform dan static untuk anggota bersama.